

Predictive Problem Set 7

Jayani Das 701

2026-03-20

PROBLEM SET 7: Shrinkage Methods

Comparison of least squares, ridge regression, LASSO and Elastic Net

You would be working with Hitters data set available in the ISLR library. You have to predict the Salary based on all available variables. First make sure that you have deleted all rows with one or more missing values.

```
library(ISLR)
data(Hitters)
Hitters <- na.omit(Hitters)
View(Hitters)
```

1. Split the data set into a training set and a test set randomly keeping 100 observations in test set and remaining in the training set.

```
set.seed(123) # for reproducibility
train_index <- sample(1:nrow(Hitters), size = nrow(Hitters) - 100, replace = FALSE)
train_data <- Hitters[train_index, ]
test_data <- Hitters[-train_index, ]
```

2. Train the model using least squares and report the train MSE and test MSE.

```
lm_model <- lm(Salary ~ ., data = train_data)
train_predictions <- predict(lm_model, newdata = train_data)
test_predictions <- predict(lm_model, newdata = test_data)
train_mse <- mean((train_data$Salary - train_predictions)^2)
test_mse <- mean((test_data$Salary - test_predictions)^2)
train_mse

## [1] 73584.77

test_mse

## [1] 146030.2
```

3. Fit ridge regression models on the training set, with grid values of λ . Report the following:
 - (i) the dimension of the coefficient matrix so obtained.
 - (ii) The l_2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ . Comment on the output.
 - (iii) Test MSE corresponding to these two choices of λ .

```
library(glmnet)
```

```

## Loading required package: Matrix

## Loaded glmnet 4.1-10

x_train <- model.matrix(Salary ~ ., data = train_data)[, -1]
y_train <- train_data$Salary
x_test  <- model.matrix(Salary ~ ., data = test_data)[, -1]
y_test  <- test_data$Salary
lambda_grid <- 10^seq(10, -2, length = 100)
ridge_model <- glmnet(x_train, y_train, alpha = 0, lambda = lambda_grid)
coef_matrix <- coef(ridge_model)
dim(coef_matrix)

## [1] 20 100

l2_norm_50 <- sqrt(sum(coef_matrix[, 50]^2))
l2_norm_60 <- sqrt(sum(coef_matrix[, 60]^2))
l2_norm_50

## [1] 379.4961

l2_norm_60

## [1] 78.6341

ridge_predictions_50 <- predict(ridge_model, s = lambda_grid[50], newx =
x_test)
ridge_predictions_60 <- predict(ridge_model, s = lambda_grid[60], newx =
x_test)

test_mse_50 <- mean((y_test - ridge_predictions_50)^2)
test_mse_60 <- mean((y_test - ridge_predictions_60)^2)
test_mse_50

## [1] 135323.5

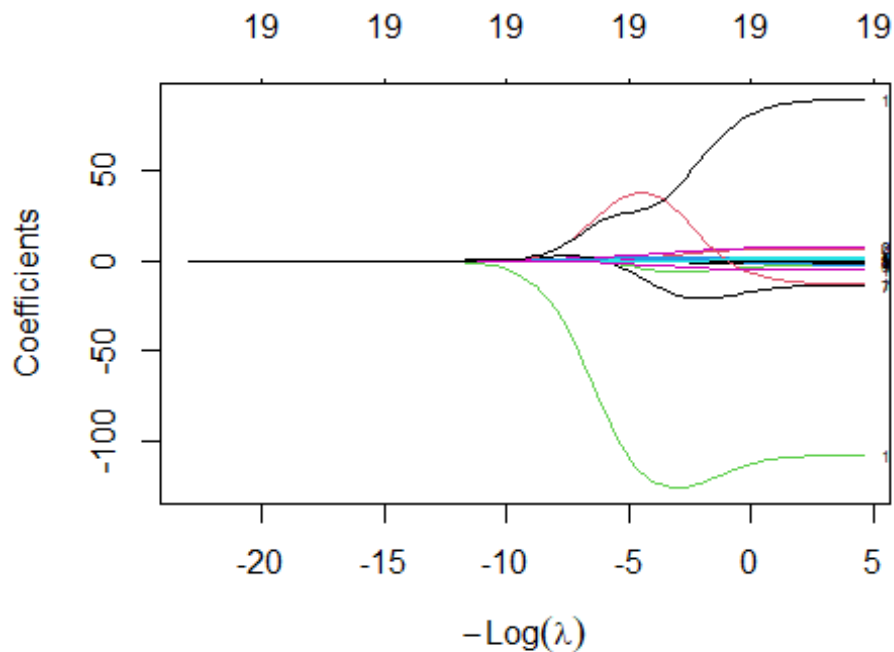
test_mse_60

## [1] 126500.1

```

4. Make a line plot (for all the predictors) of the standardized coefficients against the grid value of λ chosen by you. Interpret the graph.

```
plot(ridge_model, xvar = "lambda", label = TRUE)
```



#####

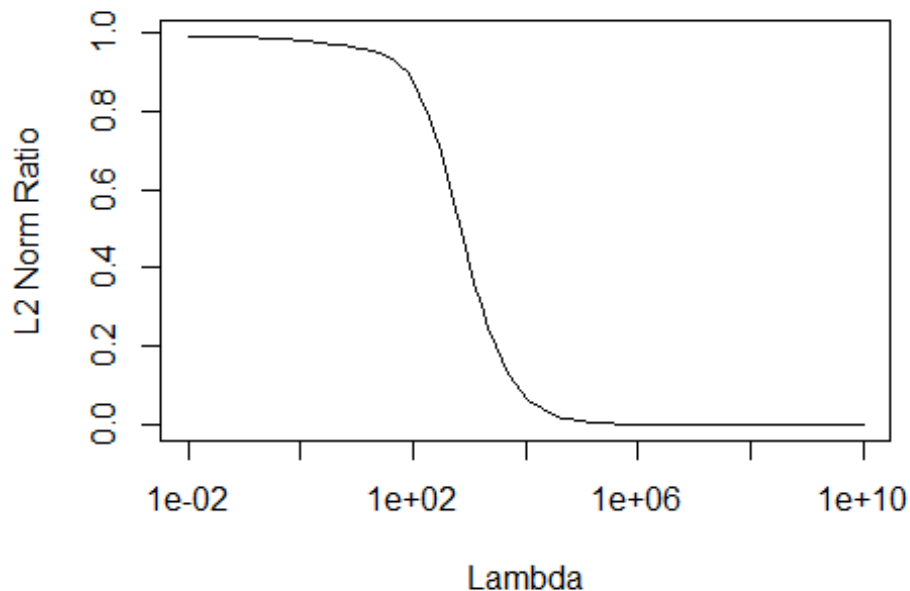
Interpretation:

As λ increases, the coefficients shrink towards zero. The plot shows how each predictor's coefficient changes with different λ values. Predictors with coefficients that shrink to zero faster are less influential in the model. This visualization helps identify which predictors are more important and how regularization affects them.

5. How would the graph in (4) change if instead of λ you plot on the x axis the ratio of l2 norm of the ridge regression coefficient to the least square regression coefficient.

```
# Calculate L2 norm of least squares coefficients
lm_coef = coef(lm_model)[-1] # Exclude intercept
lm_norm = sqrt(sum(lm_coef^2))
# Calculate L2 norm of ridge coefficients for each lambda
ridge_norms = apply(coef(ridge_model)[-1, ], 2, function(coef)
sqrt(sum(coef^2)))
# Calculate ratio
ratio = ridge_norms / lm_norm
# Plot ratio against lambda
plot(lambda_grid, ratio, type = "l", log = "x", xlab = "Lambda", ylab = "L2
Norm Ratio",
      main = "Ratio of Ridge L2 Norm to Least Squares L2 Norm")
```

Ratio of Ridge L2 Norm to Least Squares L2 Norm



#####

Interpretation:

The plot of the ratio of L2 norms will show how the ridge regression coefficients shrink relative to the least squares coefficients as λ increases. As λ increases, the ratio will decrease, indicating that the ridge coefficients are shrinking more compared to the least squares coefficients. This plot provides insight into the degree of regularization applied by ridge regression at different λ values.

6. Run the Ridge regression on training data by choosing λ by cross-validation. Find the test MSE for the optimal choice of λ obtained using cross validation.

```
cv_ridge = cv.glmnet(x_train, y_train, alpha = 0, lambda = lambda_grid)
best_lambda = cv_ridge$lambda.min
best_lambda
```

```
## [1] 10.72267
```

Test MSE for Ridge with best Lambda

```
pred_ridge = predict(ridge_model, s = best_lambda, newx = x_test)
mse_ridge = mean((y_test - pred_ridge)^2)
mse_ridge
```

```
## [1] 140080.1
```

7. Fit a LASSO model on the training set, with λ chosen by cross-validation. Report the test MSE, along with the number of non-zero coefficient estimates.

```
lasso_model = glmnet(x_train, y_train, alpha = 1, lambda = lambda_grid)
cv_lasso = cv.glmnet(x_train, y_train, alpha = 1, lambda = lambda_grid)
```

```

best_lambda_lasso = cv_lasso$lambda.min
best_lambda_lasso

## [1] 3.511192

# Test MSE for LASSO
pred_lasso = predict(lasso_model, s = best_lambda_lasso, newx = x_test)
mse_lasso = mean((y_test - pred_lasso)^2)
# Number of non-zero coefficients
lasso_coef = coef(lasso_model, s = best_lambda_lasso)
non_zero_coef = sum(lasso_coef != 0) - 1 # Exclude intercept
mse_lasso

## [1] 141163.6

non_zero_coef

## [1] 14

```

8. Fit an Elastic Net model on the training set, with λ_1 and λ_2 chosen by cross-validation. Report the test MSE. Also report the predictors with non-zero coefficients.

```

alpha_grid = seq(0, 1, by = 0.1)

cv_errors = numeric(length(alpha_grid))
lambda_values = numeric(length(alpha_grid))

# Loop over alpha values
for (i in seq_along(alpha_grid)) {
  cv_model = cv.glmnet(x_train, y_train, alpha = alpha_grid[i])

  cv_errors[i] = min(cv_model$cvm)
  lambda_values[i] = cv_model$lambda.min
}

# Select best alpha and lambda
best_index = which.min(cv_errors)
best_alpha = alpha_grid[best_index]
best_lambda = lambda_values[best_index]

best_alpha

## [1] 0

best_lambda

## [1] 31.20346

# Fit Elastic Net with best alpha and lambda
elastic_model = glmnet(x_train, y_train, alpha = best_alpha, lambda =
best_lambda)
# Test MSE for Elastic Net

```

```

pred_elastic = predict(elastic_model, s = best_lambda, newx = x_test)
mse_elastic = mean((y_test - pred_elastic)^2)
# Non-zero coefficients
elastic_coef = coef(elastic_model, s = best_lambda)
non_zero_elastic = elastic_coef[elastic_coef != 0]
mse_elastic

## [1] 139082

non_zero_elastic

## [1] 9.526718e+00 -4.971160e-01 2.553669e+00 -5.828563e+00 2.099046e+00
## [6] 6.473960e-01 4.188973e+00 -1.672935e+01 -6.067665e-05 1.397645e-01
## [11] 1.215561e+00 3.927784e-01 2.344025e-01 -2.427603e-01 3.332575e+01
## [16] -1.250328e+02 1.817195e-01 -1.129276e-02 -2.949104e+00 3.433251e+01

```

9. Compare the test MSE 's of least squares regression, Ridge regression, LASSO and ELASTIC NET and comment on how accurately we can predict the Salary of a baseball player.

```

test_mse

## [1] 146030.2

test_mse_50

## [1] 135323.5

test_mse_60

## [1] 126500.1

mse_ridge

## [1] 140080.1

mse_lasso

## [1] 141163.6

mse_elastic

## [1] 139082

```

Interpretation:

The test MSE values for least squares regression, ridge regression, LASSO, and Elastic Net provide insight into the predictive performance of each model. Generally, we expect the regularized models (ridge, LASSO, Elastic Net) to have lower test MSE compared to least squares regression due to their ability to reduce overfitting. The model with the lowest test MSE is considered the best for predicting the Salary of a baseball player. The specific values will indicate how well each model captures the underlying relationship between the predictors and the response variable.